

Detailed Summary: Pandas-III

1. Session Focus

This session covers essential DataFrame operations in pandas:

- Handling missing values
- Statistical operations (mean, median, mode, max, min, sum, count)
- Frequency analysis using value_counts
- Data aggregations using aggregate and groupby
- Sorting DataFrames with sort_values
- Finding index of max values using idxmax
- Grouping data using GROUP BY functions
- Altering the index using set_index and reset_index
- Exporting DataFrames to CSV and Excel formats

2. Statistical Operations

Statistical operations help summarize and describe data patterns. These include:

Mean

Calculates the average of numeric columns. Non-numeric columns like dates can cause errors or misleading results.

```
# Mean of all numeric columns
df.mean()

# Mean of specific column
df['paid'].mean()

# Mean of only numeric columns
df.select_dtypes(include=['number']).mean()
```

Median

Returns the middle value of numeric columns. Useful for skewed distributions.

```
df.median()
df['amount'].median()
```

Mode

Returns the most frequent value(s). Useful for categorical data.

```
df.mode()
```

```
df['gender'].mode()
```

Max & Min

Returns the maximum and minimum values. Use `numeric_only=True` to avoid issues with non-numeric columns.

```
df.max(numeric_only=True)
df.min(numeric_only=True)
```

Sum

Returns the sum of values. Avoid summing identifiers like 'id'.

```
df.sum(numeric_only=True)
```

Count

Counts non-null values in each column.

```
df.count()
```

3. Frequency Analysis

Use `value_counts()` to compute frequency of unique values in a column. Useful for categorical data analysis.

```
# Frequency of companies
df['company'].value_counts()

# Frequency of gender
df['gender'].value_counts()
```

4. Data Aggregations

Use `aggregate()` to apply one or more functions to a DataFrame or Series. Combine with `groupby` for grouped summaries.

```
# Single aggregation
df.agg('max')

# Multiple aggregations
df['cost'].agg(['max', 'min'])

# Grouped aggregation
df.groupby('company')['cost'].agg(['mean', 'median', 'count'])
```

5. Sorting DataFrames

Use `sort_values()` to sort by one or more columns. Control order using `ascending` parameter.

```
# Sort by one column
```

```
df.sort_values(by='amount')

# Sort descending
df.sort_values(by='amount', ascending=False)

# Sort by multiple columns
df.sort_values(by=['company', 'cost'], ascending=[True, False])
```

6. idxmax Function

Returns the index of the first occurrence of the maximum value. Useful for locating top records.

```
# Index of max paid amount
df['paid'].idxmax()

# Index of max cost in products
df['cost'].idxmax()
```

7. GROUP BY Functions

Split-Apply-Combine strategy:

- Split data into groups
- Apply aggregation functions
- Combine results into a summary

```
# Group by company
grouped = df.groupby('company')
grouped['cost'].mean()

# Multiple aggregations
grouped['cost'].aggregate(['mean', 'count'])

# Group by date
df.groupby('purch_date')['paid'].sum()
```

8. Altering Index

Use `set_index()` to change the index to a meaningful column. Use `reset_index()` to revert to default.

```
# Set 'id' as index
df.set_index('id', inplace=True)

# Reset to default index
df.reset_index(inplace=True)

# Set email as index
df.set_index('email', inplace=True)
```

9. Pivot Tables

Create multi-dimensional summaries using `pivot_table()`. Useful for cross-tabulations.

```
pd.pivot_table(data=df,
                values='amount',
                index='purch_date',
                columns='product_num',
                aggfunc='sum',
                fill_value=0)
```

10. Exporting DataFrames

Export DataFrames to CSV or Excel using `to_csv()` and `to_excel()`.

```
# Export to CSV
df.to_csv('output.csv', index=False)

# Export to Excel
df.to_excel('output.xlsx', index=False, sheet_name='Summary')
```

11. Key Takeaways

- Use statistical operations to summarize data
- Use `value_counts` for categorical analysis
- Use `aggregate` and `groupby` for grouped summaries
- Use `sort_values` and `idxmax` for ordering and locating top values
- Use `set_index/reset_index` to manage DataFrame structure
- Use `pivot_table` for advanced summaries
- Export data for reporting or sharing

12. Why This Matters

These operations are foundational for:

- Data cleaning and preparation
- Exploratory data analysis
- Feature engineering for machine learning
- Creating reports and dashboards

Mastering these tools enables efficient and effective data analysis workflows.